TrustVault: Automated Testing Documentation

TESTING | Snovasys Fresher Hiring Challenge 2026

1. Overview

TrustVault is a document sharing application that protects private files with login, document permissions and an audit log. The backend is written in Node.js and Express, and MySQL stores the users, the document details, the permissions and the audit records.

This file explains how I test the backend. The tests use Jest and Supertest. Jest runs the tests and Supertest sends real HTTP requests to the Express app, so the tests go through the same routes and middleware that a real user would go through.

The main goal of the tests is simple. They try to do things that a user is not allowed to do, and they expect the server to say no. A test that only checks the happy path tells very little about security, so most of my tests are of the second kind.

2. What the tests check

- A user who is not logged in cannot use protected endpoints.
- A logged-in user can reach only the documents that they own or that were shared with them.
- One user cannot read the private document of another user.
- One user cannot download the private document of another user.
- A user without the admin role cannot open the audit log.
- Refused attempts are handled safely and, where the audit code covers them, are written to the audit log.
- The API answers with the correct HTTP status codes.

3. Tools

Tool	Use
Jest	The JavaScript test framework. It runs the test files and reports passed and failed tests.
Supertest	Sends HTTP requests to the Express app and checks the responses.
MySQL	Stores the application data, the permissions and the audit logs that the tests read.

4. Test cases

Test case 1: No login on a protected document endpoint

Aim. Check that a request without a valid token is refused.

Steps. Send a request to a protected document endpoint with no Authorization header, then look at the response.

Expected result. HTTP 401 Unauthorized.

Why it matters. It stops anonymous users from reaching protected data.

Test case 2: Reading another user's document

Aim. Check that one user cannot read a document owned by another user without permission.

Steps. Create two test users, User A and User B. Create a private document that belongs to User A. Log in as User B and ask for User A's document without any permission being given.

Expected result. HTTP 403 Forbidden, as the authorization rules say.

Why it matters. It protects the content of private documents.

Test case 3: Downloading another user's document

Aim. Check that one user cannot download another user's private file.

Steps. Create a document that belongs to User A. Log in as User B and try to download it without permission.

Expected result. HTTP 403 Forbidden.

Why it matters. Knowing or guessing a document ID must never be enough to get the file.

Test case 4: Audit log without login

Aim. Check that the audit log endpoint refuses a request with no token.

Steps. Send a request to the admin audit log endpoint without a token.

Expected result. HTTP 401 Unauthorized.

Why it matters. The audit records are sensitive and must not be open to anonymous users.

Test case 5: Audit log with a normal user

Aim. Check that a normal user cannot open the admin audit log.

Steps. Log in as a user without the admin role and request the audit log endpoint.

Expected result. HTTP 403 Forbidden.

Why it matters. It shows that the role check is done on the server.

Test case 6: Allowed access still works

Aim. Check that the restrictions do not block people who should have access.

Steps. Create a document, give permission to a second test user, log in as that user and request the document.

Expected result. The request succeeds, as long as the document exists and the permission is valid.

Why it matters. A system that blocks everyone is also broken. Legitimate sharing must keep working.

Test case 7: Denied attempt is recorded

Aim. Check that a refused attempt leaves a record.

Steps. Log in as a user with no permission, try to read or download a private document, and then look at the audit log.

Expected result. The request is denied and a denied entry is saved, where the audit code covers that action.

Why it matters. It gives a trace of who tried to get in.

5. Tests I should add

The seven cases above are the base. The challenge says that the tests should also follow my own threat list, so these are the next ones that I plan to cover:

- Removing access. Give access, confirm it works, remove it, and confirm that the very next request is refused. This is the test for the rule that taking access away must work immediately.
- Edit and delete. A user with read only permission must not be able to edit or delete the document.
- Permission changes. A user who is not the owner must not be able to grant or remove access.
- Uploads. A wrong file type and a file larger than 5 MB must be rejected.

- Search and listing. A user must not see names of documents that they cannot access.
- Token problems. A missing, wrong or expired token must give 401.

6. Checking that the tests are real

A passing test means nothing if the test cannot fail. The challenge team will take one permission check out of my code and run my tests again, and if the tests still pass then they were not testing anything. I can do the same check myself before submitting:

1. Run the full test suite and confirm that everything passes.
2. Open the document route and comment out the ownership or permission check, only on my own computer.
3. Run the tests again. At least one test should fail now, and the failing test should be the cross-user read or download test.
4. Put the check back and run the tests once more to confirm that they pass.
5. Do the same with the admin role check on the audit log endpoint.

If the tests still pass after step 3, that test is weak and must be rewritten before I submit.

7. How to run the tests

Requirements are Node.js with npm, the project dependencies, and a test configuration that matches the database and the environment variables of the application.

Open PowerShell, go to the server folder and install the dependencies:

```
cd TrustVault/server
npm install
```

Then run the tests:

```
npm test
```

Jest prints how many tests passed and failed. I only report the tests as passed when Jest says so. I do not write a result in any document before I have seen it in the terminal.

8. Safety rules for testing

- Use separate test accounts. The demo accounts of the live site are not used in the tests.
- Never run tests that delete or change data against the production database.
- Do not write real passwords, JWT secrets or database credentials inside the test files.
- Clean the test records after the tests where it is needed, so that old test data does not affect the next run.

- Keep the login and permission tests inside the automated test suite so that they run every time.

9. Limits of these tests

The tests cover selected cases and not every possible attack. Passing tests do not prove that the application is secure. They only prove the cases that were written. This is why THREATS.md also lists the weak points that I know about and have not closed.

10. Conclusion

The most important tests are the cross-user read and download tests. They show that having a document ID is not enough to get the document, and that the server decides every time. Testing this automatically means that if I break a permission check later by mistake, a test will tell me before a user does.